Corso di Cybersecurity – Epicode

Report dell'Esercitazione

Analisi statica di un eseguibile sospetto con PEStudio

Librerie importate, funzioni segnalate e sezioni del file notepad-classico.exe

A cura di:

Nicola Madonna

Epicode

4 agosto 2026

Indice

1 Riassunto 2
2 Obiettivi 2
3 Configurazione del Laboratorio 3
4 Analisi dei Dati Raccolti 4
4.1 Librerie importate 4
4.2 Funzioni segnalate da PEStudio 5
4.3 Sezioni del PE 6
4.4 Anomalie riscontrate 7
5 Conclusioni 9

1 Riassunto

Questo report descrive l'analisi statica del file `notepad-classico.exe`, un campione presente nella cartella *Malware* della macchina virtuale FlareVM. Il file è stato esaminato con **PEStudio**, che legge la struttura del formato Portable Executable (PE) senza eseguire il programma. A prima vista il file si presenta come il Blocco note di Windows: metadati e librerie importate sono quelli di un editor di testo. La struttura delle sezioni però non è quella di un eseguibile legittimo: c'è una seconda sezione `.text`, scrivibile ed eseguibile, che occupa più del 60% del file e contiene il punto di ingresso del programma, e le sezioni sono sei, con una seconda sezione di risorse al posto di quella originale. Il quadro è compatibile con un Blocco note modificato (trojanizzato) o protetto da un packer, ipotesi che andrebbe confermata con un'analisi dinamica.

2 Obiettivi

- Individuare le librerie importate dal file e descrivere il ruolo di ciascuna.
- Esaminare le funzioni che PEStudio segnala come potenzialmente sospette.
- Individuare le sezioni del PE e riconoscere eventuali anomalie.
- Formulare un'ipotesi sulla natura del file sulla base della sola analisi statica.

3 Configurazione del Laboratorio

L'analisi è stata svolta sulla macchina virtuale FlareVM, isolata dalla rete, aprendo il file in PEStudio. La Tabella 1 riassume le informazioni generali sul campione mostrate dallo strumento; gli hash del file, calcolati con HashMyFiles e confermati da PEStudio, sono riportati subito dopo la tabella.

Tabella 1: Informazioni generali sul file analizzato.

Proprietà	**Valore**
Nome file	notepad-classico.exe
Percorso	C:\Users\FlareVm\Desktop\Malware\
Tipo	eseguibile Windows a 32 bit, interfaccia grafica (GUI)
Dimensione	289.280 byte
Entropia del file	6,273
Nome originale (version info)	NOTEPAD.EXE
Descrizione (version info)	Blocco note

```
SHA-256: d2e6c9f9273663f3218bcd7cbfb3b6f599fbce7a4ba986f9bbff77e3603988f2
MD5:     8a00a5c59ac157754ca575d721bcf960
```

4 Analisi dei Dati Raccolti

4.1 Librerie importate

Il file importa **9 librerie** (DLL) per un totale di **201 funzioni** (Fig. 1). Nessuna libreria è segnalata come sospetta di per sé; la Tabella 2 ne descrive il ruolo.

library (9)	flag (0)	type	imports (201)	description
comdlg32.dll	-	Implicit	9	Common Dialogs Library
SHELL32.dll	-	Implicit	4	Windows Shell Library
WINSPOOL.DRV	-	Implicit	3	Windows Spooler Driver
COMCTL32.dll	-	Implicit	1	Common Controls Library
msvcrt.dll	-	Implicit	22	Microsoft C Runtime Library
ADVAPI32.dll	-	Implicit	7	Advanced Windows 32 Base API
KERNEL32.dll	-	Implicit	57	Windows NT BASE API Client
GDI32.dll	-	Implicit	24	GDI Client Library
USER32.dll	-	Implicit	74	Multi-User Windows USER API Client Library

Figura 1: Librerie importate da notepad-classico.exe secondo PEStudio: 9 DLL e 201 funzioni.

Tabella 2: Ruolo delle librerie importate.

Libreria	**Funzioni**	**Ruolo**
COMDLG32.dll	9	Finestre di dialogo comuni: apri e salva file, stampa, scelta del carattere, trova e sostituisci. Tipiche di un editor di testo.
SHELL32.dll	4	Interazione con la shell di Windows, tra cui il trascinamento dei file da Esplora risorse verso l'applicazione.
WINSPOOL.DRV	3	Interfaccia con lo spooler di stampa, per inviare documenti alla stampante.
COMCTL32.dll	1	Controlli comuni dell'interfaccia; qui una sola funzione, per la barra di stato.
msvcrt.dll	22	Libreria di runtime del C: memoria, stringhe, tempo e terminazione del processo. Presente in quasi tutti i programmi scritti in C/C++.
ADVAPI32.dll	7	API avanzate di Windows, in particolare accesso al registro e gestione della sicurezza. Da tenere d'occhio, perché la scrittura nel registro è un modo comune per ottenere persistenza.
KERNEL32.dll	57	API di base del sistema: file, memoria, processi e caricamento di moduli. Presente in ogni eseguibile Windows.
GDI32.dll	24	Disegno di testo, caratteri e oggetti grafici a schermo e in stampa.
USER32.dll	74	Finestre, menu, messaggi, tastiera, mouse e appunti: la base di ogni applicazione con interfaccia grafica.

4.2 Funzioni segnalate da PEStudio

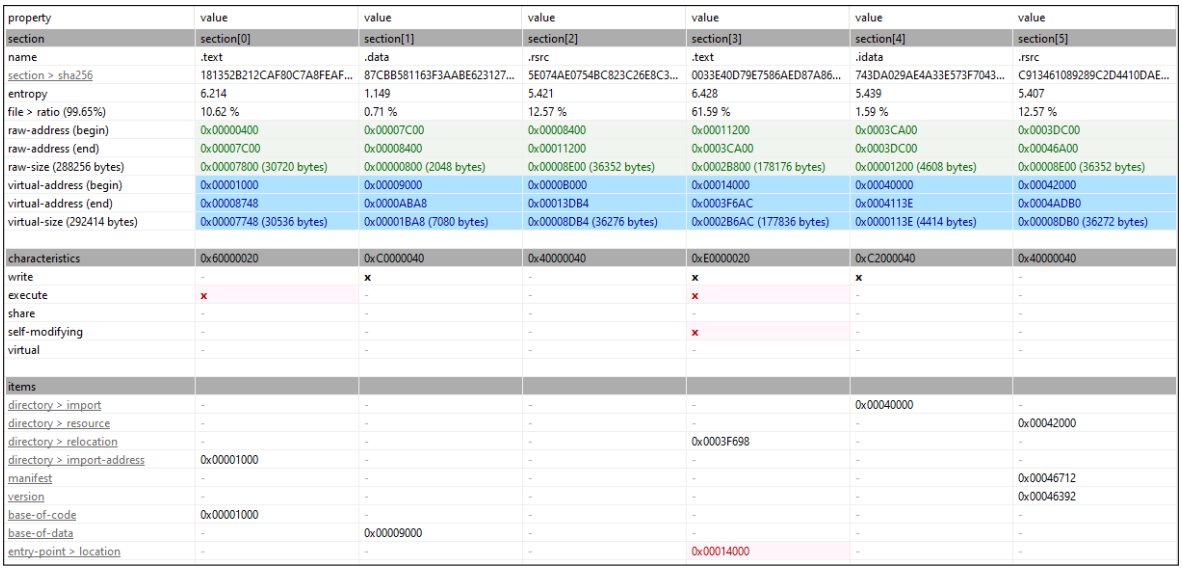
PEStudio segnala 18 delle 201 funzioni importate, perché sono spesso presenti nei malware. La Tabella 3 le raggruppa per categoria. Gli elenchi completi delle importazioni sono disponibili come immagini nella cartella *foto_utili*.

Tabella 3: Le 18 funzioni importate segnalate da PEStudio.

Categoria	**Funzioni**	**Uso possibile**
Registro (ADVAPI32)	`RegCreateKeyW`, `RegSetValueExW`	Creare chiavi e scrivere valori nel registro, anche per la persistenza.
Processi e thread (KERNEL32)	`GetCurrentProcess`, `GetCurrentProcessId`, `GetCurrentThreadId`	Ottenere informazioni sul proprio processo, ad esempio prima di manipolarlo.
File system (KERNEL32)	`FindFirstFileW`, `WriteFile`, `DeleteFileW`	Enumerare, scrivere e cancellare file.
Mappatura della memoria (KERNEL32)	`MapViewOfFile`, `UnmapViewOfFile`	Mappare file o memoria condivisa nello spazio del processo.
Finestre ed eventi (USER32)	`GetForegroundWindow`, `GetDesktopWindow`, `SetWinEventHook`, `UnhookWinEvent`, `WinHelpW`	Sapere quale finestra è in primo piano e ricevere notifiche sugli eventi dell'interfaccia.
Appunti (USER32)	`OpenClipboard`, `CloseClipboard`, `IsClipboardFormatAvailable`	Leggere e scrivere il contenuto degli appunti.

Queste funzioni vanno lette nel contesto. Per un vero Blocco note quasi tutte hanno una spiegazione legittima: copia e incolla usa gli appunti, il salvataggio usa `WriteFile`, le impostazioni dell'editor sono salvate nel registro. Da sole non bastano quindi a dichiarare il file malevolo. La coppia `SetWinEventHook` / `UnhookWinEvent` è la meno ovvia per un editor, perché permette di ricevere notifiche sugli eventi dell'interfaccia di altre applicazioni, e merita un approfondimento in fase dinamica.

Tra le funzioni **non** segnalate ce ne sono comunque alcune utili all'analisi: `LoadLibraryA` e `GetProcAddress` consentono di caricare librerie e risolvere funzioni durante l'esecuzione, cioè di usare API che non compaiono nella tabella delle importazioni; `CreateFileMappingW` completa la mappatura di file in memoria insieme a `MapViewOfFile`.



property	value	value	value	value	value	value
section	section[0]	section[1]	section[2]	section[3]	section[4]	section[5]
name	.text	.data	.rsrc	.text	.idata	.rsrc
section > sha256	181352B212CAF80C7A8FEAF...	87CBB581163F3AABE623127...	5E074AE0754BC823C26E8C3...	0033E40D79E7586AED87A86...	743DA029AE4A33E573F7043...	C913461089289C2D4410DAE...
entropy	6.214	1.149	5.421	6.428	5.439	5.407
file > ratio (99.65%)	10.62 %	0.71 %	12.57 %	61.59 %	1.59 %	12.57 %
raw-address (begin)	0x00000400	0x00007C00	0x00008400	0x00011200	0x0003CA00	0x0003DC00
raw-address (end)	0x00007C00	0x00008400	0x00011200	0x0003CA00	0x0003DC00	0x00046A00
raw-size (288256 bytes)	0x00007800 (30720 bytes)	0x00000800 (2048 bytes)	0x00008E00 (36352 bytes)	0x0002B800 (178176 bytes)	0x00001200 (4608 bytes)	0x00008E00 (36352 bytes)
virtual-address (begin)	0x00001000	0x00009000	0x0000B000	0x00014000	0x00040000	0x00042000
virtual-address (end)	0x00008748	0x0000ABA8	0x00013DB4	0x0003F6AC	0x0004113E	0x0004ADB0
virtual-size (292414 bytes)	0x00007748 (30536 bytes)	0x00001BA8 (7080 bytes)	0x00008DB4 (36276 bytes)	0x0002B6AC (177836 bytes)	0x0000113E (4414 bytes)	0x00008DB0 (36272 bytes)
characteristics	0x60000020	0xC0000040	0x40000040	0xE0000020	0xC2000040	0x40000040
write	-	x	-	x	x	-
execute	x	-	-	x	-	-
share	-	-	-	-	-	-
self-modifying	-	-	-	x	-	-
virtual	-	-	-	-	-	-
items						
directory > import	-	-	-	-	0x00040000	-
directory > resource	-	-	-	-	-	0x00042000
directory > relocation	-	-	-	0x0003F698	-	-
directory > import-address	0x00001000	-	-	-	-	-
manifest	-	-	-	-	-	0x00046712
version	-	-	-	-	-	0x00046392
base-of-code	0x00001000	-	-	-	-	-
base-of-data	-	0x00009000	-	-	-	-
entry-point > location	-	-	-	0x00014000	-	-

Figura 2: Sezioni del PE in PEStudio: la seconda .text (section[3]) è scrivibile, eseguibile, auto-modificante e contiene il punto di ingresso; la section[5] è una seconda .rsrc.

4.3 Sezioni del PE

Il file contiene **6 sezioni** (Fig. 2). La Tabella 4 ne riassume le caratteristiche principali.

Tabella 4: Caratteristiche delle sezioni del file.

Sezione	**Entropia**	**Dim. raw (byte)**	**Quota del file**	**Permessi**	**Note**
.text (section[0])	6,214	30.720	10,62%	lettura, esecuzione	Codice; contiene la Import Address Table e il base-of-code. Nessuna anomalia.
.data (section[1])	1,149	2.048	0,71%	lettura, scrittura	Dati inizializzati; entropia molto bassa. Nessuna anomalia.
.rsrc (section[2])	5,421	36.352	12,57%	sola lettura	Sezione di risorse di 36.352 byte a cui non punta nessuna directory delle risorse: PEStudio non vi associa né risorse né manifest né version info.
.text (section[3])	6,428	178.176	61,59%	**lettura, scrittura, esecuzione**	**Anomala**: secondo .text, flag *self-modifying*, contiene il punto di ingresso (0x00014000) e la directory di rilocazione.
.idata (section[4])	5,439	4.608	1,59%	lettura, scrittura	Directory delle importazioni.
.rsrc (section[5])	5,407	36.352	12,57%	sola lettura	Seconda sezione di risorse: contiene la directory delle risorse, il manifest e la version info.

4.4 Anomalie riscontrate

Il dato più rilevante è la sezione `section[3]`. Un eseguibile compilato normalmente ha una sola sezione `.text`; qui ce n'è una seconda con lo stesso nome, ed è anomala per diversi motivi:

- ha permessi di **scrittura ed esecuzione** insieme (RWX) e PEStudio la marca come *self-modifying*: è una combinazione tipica del codice che si decifra o si ricostruisce in memoria durante l'esecuzione;
- il **punto di ingresso** del programma (0x00014000) coincide con l'inizio di questa sezione e non si trova nella prima `.text`, dove risiede il codice originale: all'avvio viene eseguito per primo il codice della sezione aggiunta;
- è la sezione più grande del file: con 178.176 byte occupa il 61,59% del totale, quasi sei volte il codice della `.text` originale.

Un secondo dato riguarda le sezioni di risorse. Ci sono due sezioni `.rsrc` (section[2] e section[5]) con la stessa dimensione raw, 36.352 byte, ed entropia quasi uguale (5,421 e 5,407), ma con contenuto diverso, come mostrano i loro SHA-256. La directory delle

risorse (0x00042000), il manifest e la version info si trovano nella seconda; la prima non ha alcun elemento associato. Anche la directory delle importazioni è in una sezione `.idata` in coda al file (0x00040000), mentre la Import Address Table è nella prima `.text`. Le prime tre sezioni (`.text`, `.data`, `.rsrc`) hanno la struttura di un Blocco note originale; le ultime tre (`.text`, `.idata`, `.rsrc`) sembrano aggiunte in un secondo momento, come accade quando un file viene modificato o ricostruito da un tool di patching o da un packer.

Non c'è invece nessun overlay: PEStudio riporta la voce *overlay* come non applicabile e le sezioni coprono il 99,65% del file, cioè tutti i 288.256 byte oltre ai 1.024 byte di header.

5 Conclusioni

Superficialmente `notepad-classico.exe` si presenta come il Blocco note di Windows: version info, descrizione e quasi tutte le librerie importate sono coerenti con un editor di testo. È un caso tipico di *masquerading*, in cui un file malevolo imita un'applicazione nota per non destare sospetti.

La struttura interna racconta però un'altra storia: una seconda sezione `.text` scrivibile, eseguibile e auto-modificante, che contiene il punto di ingresso ed è sei volte più grande del codice originale, con in più una seconda sezione di risorse al posto di quella originale. Il quadro è compatibile con due scenari: un Blocco note **trojanizzato**, a cui è stata aggiunta una sezione con il codice malevolo eseguito all'avvio, oppure un file protetto da un **packer** che decomprime il vero payload solo in memoria, per eludere l'analisi statica e le firme antivirus.

L'analisi statica da sola non permette di scegliere tra le due ipotesi né di stabilire cosa faccia il codice. Il passo successivo è un'analisi dinamica in ambiente isolato e monitorato, osservando processi creati, scritture su file e registro, uso degli hook sugli eventi e connessioni di rete, insieme al confronto dell'hash del file con i servizi di threat intelligence.